

26-2 졸업프로젝트 중간발표
— 비주얼 노벨을 위한 월드-에이전트 인터랙티브 서사 시스템

(A World-Agent Interactive Narrative System for Visual Novels)

컴퓨터공학부 202315308 김연하

목차

I. 프로젝트 개요 및 목표

II. 팀원별 소개 및 역할

III. 요구사항 분석

IV. 프로젝트 구성도 및 서비스 모델

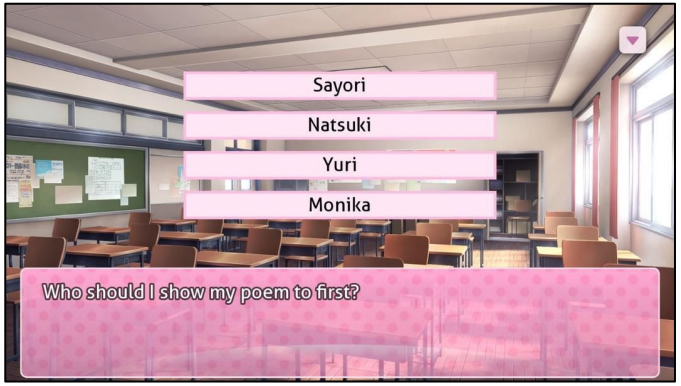
V. 시스템 구조 설계

VI. 프로토타입 개발 및 테스트 계획

VII. 추후 진행 일정

I. 프로젝트 개요 및 목표

프로젝트 배경 — Visual Novel에서의 상호작용



<선택지 기반 상호작용 예시>

비주얼 노벨(Visual Novel, VN)

: 텍스트와 일러스트를 중심으로 스토리가 진행되며,
플레이어의 선택에 따라 이야기의 전개가 달라지는 게임 장르

- 제작자가 Dialogue · Choice · Branch를 사전에 설계
- 선택에 따라 Game State와 Story Branch 변화
- 의도된 이야기 구조와 연출을 정밀하게 구성 가능
- 플레이어의 의사 표현은 사전에 정의된 선택지 범위로 제한

정교한 서사 설계가 가능한 반면, 플레이어의 상호작용은 사전에 정의된 범위로 제한되는 한계가 있음

최근 들어, 사전 정의된 선택지와 스크립트 중심의 VN 상호작용 구조를
LLM 기반 자유형 자연어 입력 및 동적 응답 생성으로 확장하려는 시도가 증가

문제 정의 — LLM 도입에 따른 기술적 과제

자유형 상호작용을 지속적인 게임 경험과 서사로 연결하기 위해,
캐릭터·세계·서사 상태를 지속적으로 추적·관리하는 구조가 필요

① 캐릭터 일관성 유지
(Character Consistency)

- ▶ 성격 · 가치관 · 말투의 일관성
- ▶ 장면과 상황이 변화해도 Persona 유지

② 장기 경험 및 관계의 지속
(Long-term Character State)

- ▶ 주요 Interaction과 관계 변화의 지속적 축적
- ▶ Memory Retrieval을 통한 과거 경험 활용

③ 게임 세계 상태의 일관성
(World State Consistency)

- ▶ World State 지속 관리
- ▶ 상호작용 결과를 이후 게임 세계에 반영

④ 의도된 서사 진행 유지
(Narrative Control)

- ▶ 현재 Story Progress 지속 관리
- ▶ Narrative Goal / Constraint 기반 진행 제어

<자유형 LLM 상호작용을 게임으로 구성하기 위한 핵심 기술 과제>

제안 시스템 및 프로젝트 목표

프로젝트 목표

플레이어와의 경험과 관계, 게임 세계의 변화를 지속적으로 반영하여 캐릭터의 장기적 일관성을 유지하고,
자유형 상호작용 속에서도 의도된 서사 진행이 가능한 Agent 기반 Interactive Narrative 시스템을 구현한다.

자유형 자연어 상호작용
(Free-form Player Interaction)

WAIN: A World-Agent Interactive Narrative System

지속적인 캐릭터 경험
(Persistent Character Experience)

기본 캐릭터 설정을 기반으로 경험·관계 변화를 누적하여, 일관성과 변화 가능성을 함께 유지

게임 세계와 연계된 상호작용
(World-Aware Interaction)

플레이어 행동과 World State 변화를 지속적으로 반영하여, 게임 세계와 상호작용의 일관성 유지

일관된 서사 진행
(Coherent Narrative Progression)

자유로운 상호작용 속에서도 현재 서사 상태와 제작 의도를 반영하여, 이야기의 진행 방향 유지

제안서 대비 주요 변경 사항

기존 제안과 달라진 두 가지 결정: State 갱신 시점 변경 및 State 변경 발생 요인 제한

① State 갱신 시점을 VN Response 이전으로 변경

BEFORE · 기존 제안서

VN Response → State Update

VN 응답 이후 State 갱신

AFTER · 현재 설계

Validation → State Commit → VN Response

검증 작업 이후 확정된 State를 반영, VN 응답은 이후에

② State 갱신 요인을 대화(Interaction)로 한정

BEFORE · 기존 제안서

State 갱신 요인 미한정

독립적인 게임 이벤트에 대한 제한 미명시

AFTER · 현재 설계

Interaction-Driven State Update

대화 턴에 의한 변경만 처리 · 독립 World Event 제외

II. 팀원별 소개 및 역할

프로젝트 담당 및 역할

김연하

- 컴퓨터공학부 4학년
- GitHub
  : github.com/Yonaim

프로젝트 역할

- 1인 프로젝트: 기획·설계·구현·테스트 전 과정 담당

관련 역량 및 경험

- 게임 엔진 프로그래머 지망
- CS · 그래픽스 · 성능 최적화를 중심으로 학습 및 개발
- 자체 게임 엔진 · 시스템 개발 경험
- Visual Novel을 비롯한 캐릭터 중심 게임 콘텐츠에 지속적인 관심

관련 활동

- 2026.03 ~ 2026.08: 크래프톤(Krafton) 정글 게임테크랩 교육 이수
- Unreal Engine 플러그인 ‘Dynamic Wet Clothes’ 개발 및 Fab 출시

III. 요구사항 분석

사용자 요구사항과 소프트웨어 요구사항의 대응 관계

사용자 요구 → 시스템 기능 및 품질 요구 | 5 UR · 12 FR · 7 NFR

User Requirements (UR)	Software Requirements (SR)
① 자유로운 자연어 상호작용 (UR-01)	입력 처리·응답 생성·VN 출력 (FR-01, 09, 10 / NFR-07)
② 캐릭터 기억 및 세계 상태 지속 (UR-02, 03)	기억·관계·세계 상태 관리 (FR-02~07, 11 / NFR-02)
③ 서사 규칙 및 정보 공개 유지 (UR-04)	서사 제약 설정·응답 검증 (FR-08, 12 / NFR-02, 06)
④ 시스템의 범용성 (UR-05)	VN 연동·모듈성·확장성 (FR-10 / NFR-01, 03)

※ 전체 UR·FR·NFR 상세 명세 및 요구사항은 부록 참고

핵심 품질 요구사항에 따른 설계 방향

핵심 비-기능 요구사항(NFR)을 WAIN의 아키텍처 설계 원칙으로 구체화

① 상태 일관성·안정성
(NFR-02, 06)

- 승인된 State 변경 유지
- 미승인 변경 및 오류 차단

② 모듈성·확장성
(NFR-01, 03)

- 모듈별 책임 분리
- 게임 콘텐츠 의존성 최소화

③ 테스트·관측 가능성
(NFR-04, 05)

- 동일 시나리오 재현
- 상호작용 · State 변경 추적

State Ownership & Validation

- State 소유권 및 관리 책임 분리
- LLM 제안과 확정 State 구분
- 검증된 State 변경만 적용

Modular Plugin Architecture

- 모듈별 책임 및 인터페이스 분리
- 게임 시스템 의존성 최소화
- Plugin 기반 재사용·확장

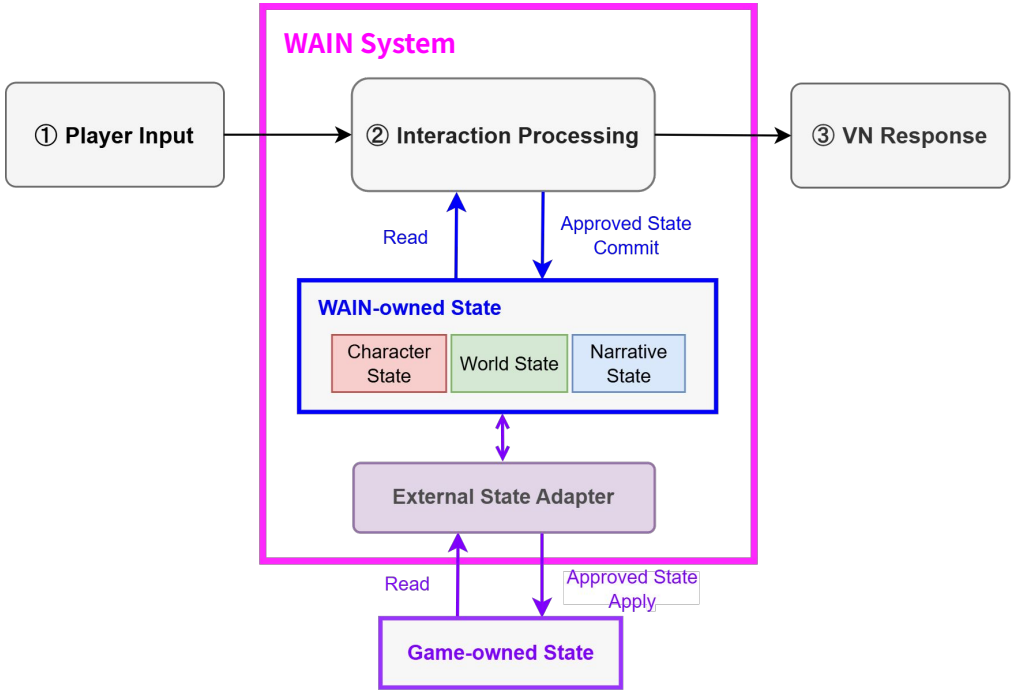
Logging & Evaluation

- 입력 · Context · 응답 기록
- Validation·Commit 이력 추적
- 동일 시나리오 기반 기능 검증

IV. 프로젝트 구성도 및 서비스 모델

시스템 동작 및 사용자 흐름

현재 State를 참조해 입력을 처리하고, 검증·확정된 State를 다음 턴에서 다시 참조



① Player Input

: 플레이어의 자유형 자연어 입력

② Interaction Processing

: WAIN · Game 소유 State 참조

→ Response 제안 → Validation

→ State Commit

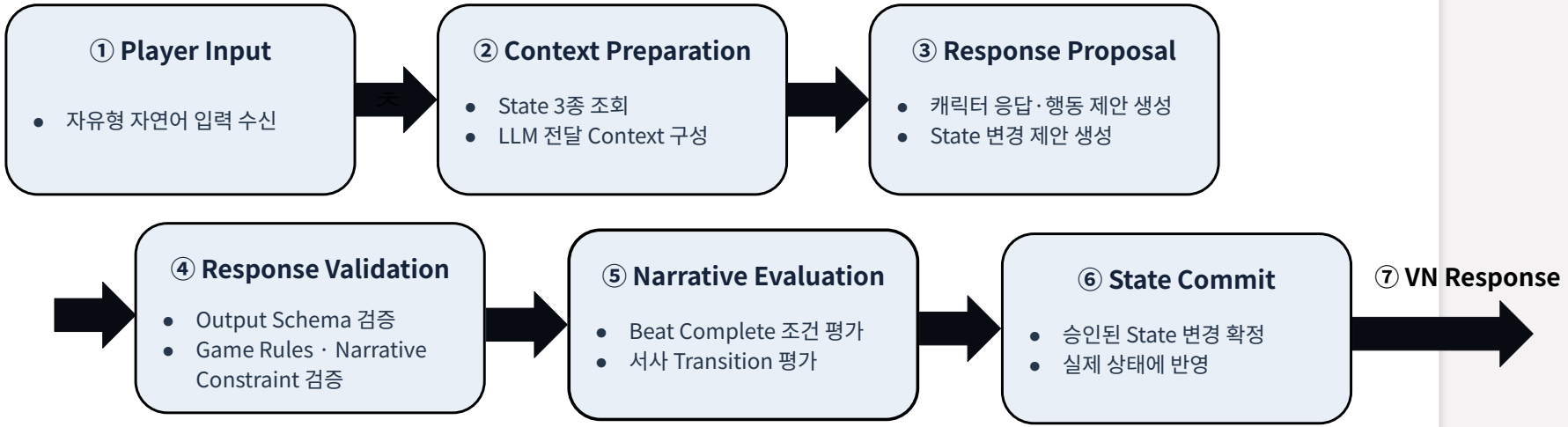
※ 갱신된 State는 다음 턴의 Context 구성에 반영

③ VN Response

: Dialogue · Action · Expression · Event 출력

Turn-Based Interaction Pipeline

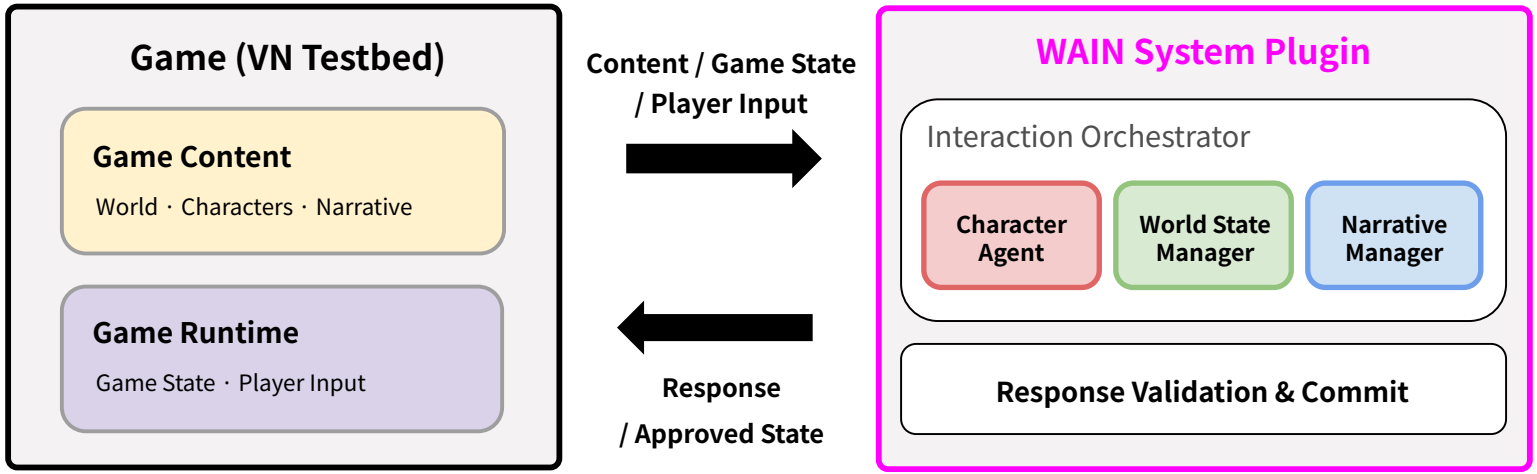
검증된 변경을 먼저 Commit한 뒤, VN 응답을 표시하고 다음 입력을 허용



- 턴 단위로 대화 진행 (Turn-Based Interaction)
- State는 대화에 의해서만 갱신됨 (Dialogue-Driven State update)

Plugin 기반 독립적 설계 및 State 연동 구조

특정 게임 콘텐츠 및 로직에 얽매이지 않는 시스템 · State별 단일 Source of Truth 유지 및 연동



State 소유권

- Game-owned → External Adapter
- WAIN-owned → Internal Store

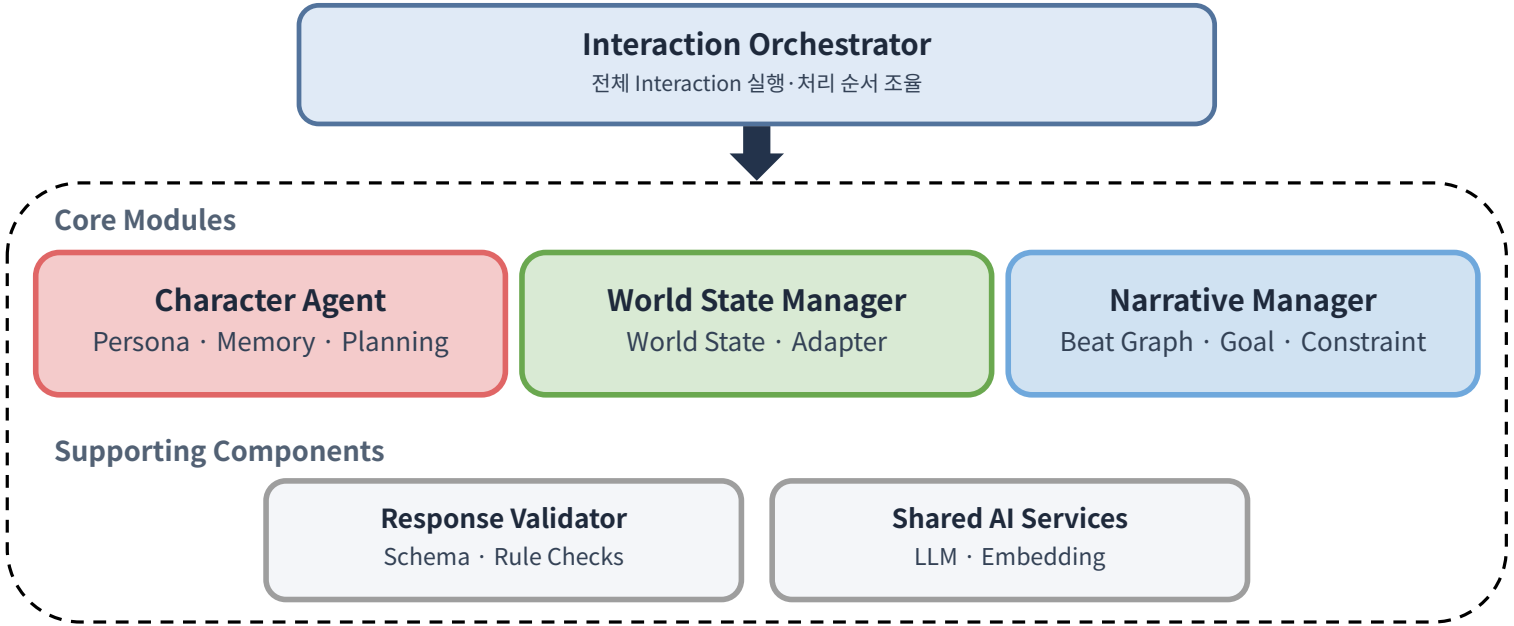
시스템 재사용성

- 게임별 Content · Adapter만 재설정하면 재사용 가능
- 공통 WAIN Plugin 처리 로직 재사용

V. 시스템 구조 설계

WAIN System Architecture

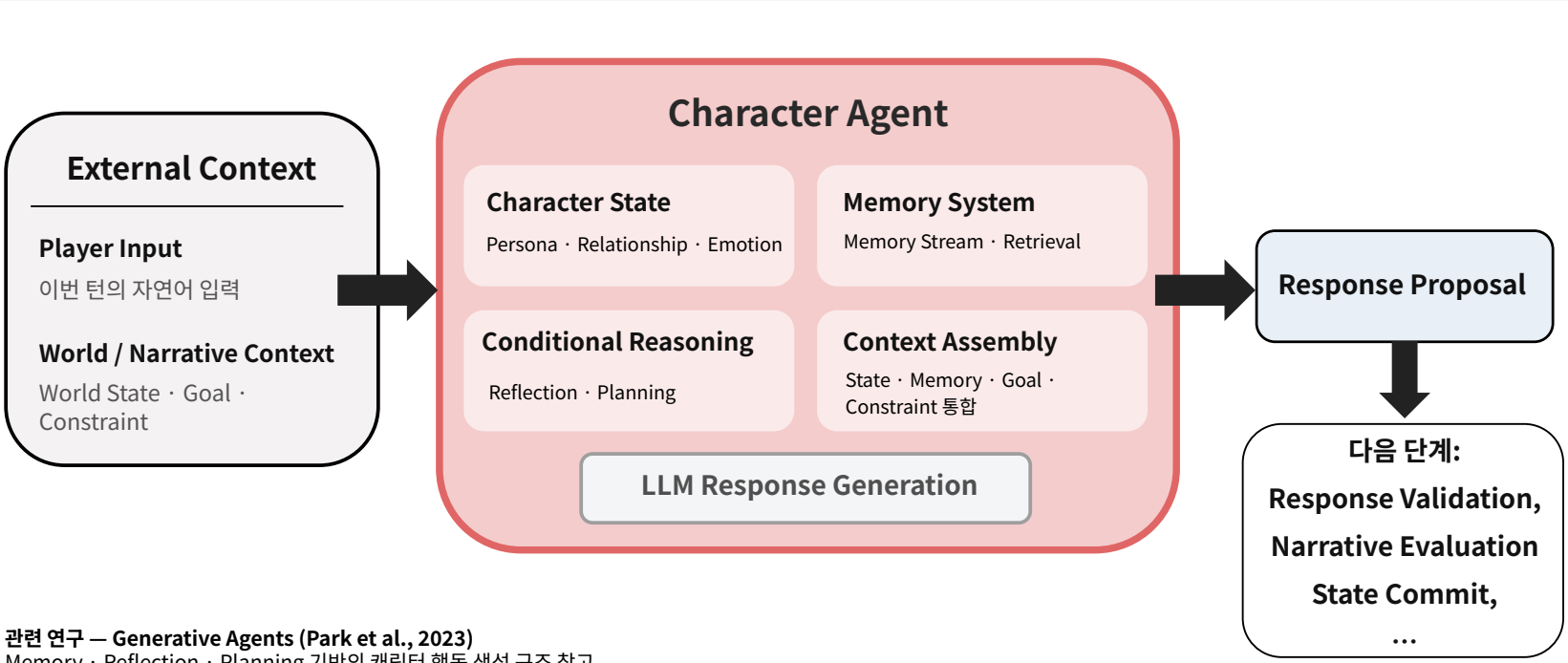
Orchestrator가 전체 파이프라인 및 상호작용을 조율 · 핵심 모듈의 책임 분리



관련 연구 — IBSEN (Han et al., 2024)
Director–Actor 역할 분리 개념을 Narrative Manager·Character Agent의 책임 분리에 참고

01. Character Agent — Overview

State · Memory · Planning 기반 Context Assembly 및 Response Proposal 생성



관련 연구 — Generative Agents (Park et al., 2023)
Memory · Reflection · Planning 기반의 캐릭터 행동 생성 구조 참고

01. Character Agent — Memory Stream

3가지 Memory Type을 공통 구조로 하나의 Stream에 통합 저장 · ID 참조로 기억 간 근거 관계를 보존

Unified Memory Stream

공통 Memory Schema

Id · Type · Content · SourceMemoryIds, etc …

3 Memory Types

① **Observation**	경험·발언 등 관찰 사실 기록
② **Reflection**	관련 경험의 상위 수준 해석
③ **Plan**	Goal · Steps 기반 계획

ID-based Memory Reference

예시) Reflection이 **근거 Observation의 ID**를 참조

Reflection R-01

"플레이어는 딸기우유를 좋아한다"

SourceMemoryIds = [O-01, O-02]

근거 Memory ID 참조

Ob-01 *"첫날, 플레이어가 딸기우유 선택"*

Ob-02 *"며칠 뒤, 다시 딸기우유 선택"*

관련 연구 — Generative Agents (Park et al., 2023) 통합 Memory Stream 구조 및 ID 기반 기억 참조 방식 참고

01. Character Agent — Memory Retrieval

의미 유사도 기반 후보 검색 후, 관련성·최근성·중요도를 종합 평가하여 선정

① Query Embedding

- 현재 Query를 Embedding 벡터로 변환

→

② Semantic Search

- Cosine 유사도 기반
- 상위 N개 후보 Memory 확보

→

③ Scoring & Top-K

- **후보 Memory의 Score 평가**
- 상위 K개 → Agent Context

*각 Memory Embedding은 저장 당시 생성됨

Memory Scoring 계산식

※ 정규화 방식 · 가중치 · 후보 수(N) · 최종 선택 수(K) · Embedding 모델은 추후 결정

$$S(m,q) = \alpha R(m,q) + \beta C(m) + \gamma I(m)$$

R = Relevance
Cosine 유사도 · 의미 관련성

C = Recency
경과 Turn 수에 따른 최신성 감쇠

I = Importance
Memory 생성시 부여한 중요도

관련 연구 — Generative Agents (Park et al., 2023) Embedding 기반 관련 기억 검색 · Memory Scoring 방식 참고

01. Character Agent — Reflection

Importance 누적값을 기준으로 Reflection을 실행하고, 관련 기억에서 Insight 생성

※ 원 논문의 설정값은 변수로 표기했으며, WAIN의 실제 적용값은 추후 확정

① Reflection Trigger

- 최근 Importance 누적값 ≥ T
- Trigger 미충족시 Reflection 생략

② Question Generation

- 최근 Memory N개를 LLM에 제공
- 생각 가치가 있는 핵심 질문 M개 생성

③ Relevant Memory Retrieval

- 각 질문을 Query로 사용
- 관련 기억을 검색

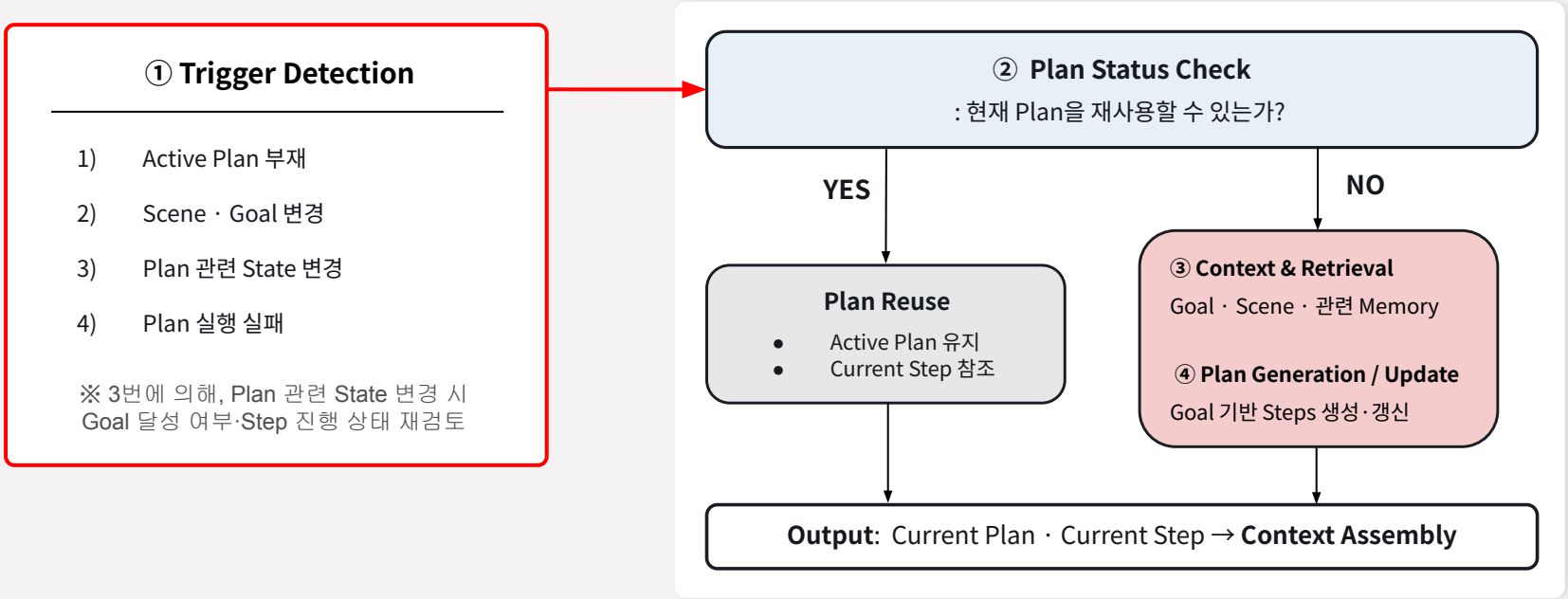
④ Insight Generation

- 각 질문당 최대 K개의 Insight 생성
- Reflection 타입 Memory로 저장

관련 연구 — Generative Agents (Park et al., 2023)
Importance 기반 Reflection 실행 조건 · 질문 생성 · Memory Retrieval · Insight 생성 과정 참고

01. Character Agent — Planning

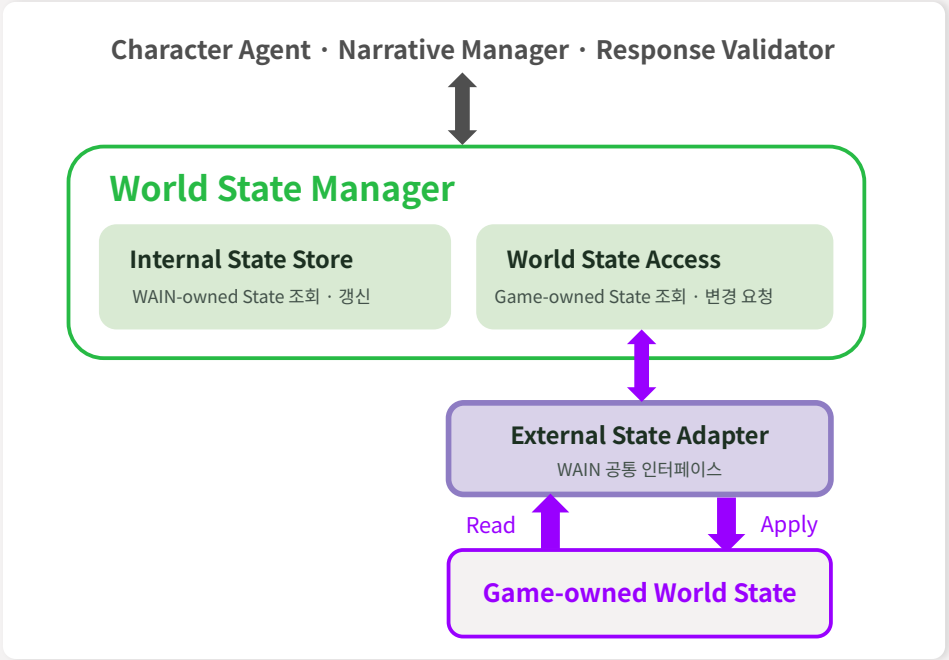
Importance 누적값을 기준으로 Reflection을 실행하고, 관련 기억에서 Insight 생성



관련 연구 — Generative Agents (Park et al., 2023)
시간 기반 Planning 구조를 Turn-Based VN 환경에 맞춰 Goal·Scene 중심으로 변형

02. World State Manager — State Ownership

WAIN-owned World State는 직접 관리 · Game-owned World State는 공통 Adapter로 접근



Ownership Principles

① Single Source of Truth

: 각 State는 원래 소유처에서 관리

② Controlled State Access

: Game-owned State는 Adapter를 통해 접근

③ Validated Changes Only

: 검증·승인된 변경만 Commit에서 반영

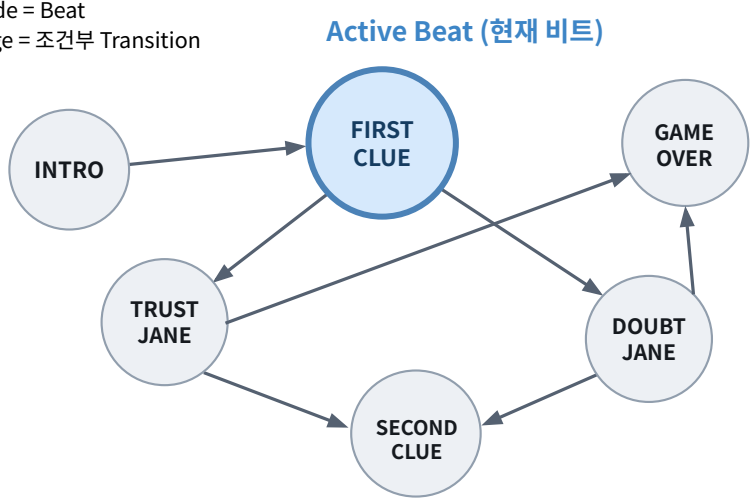
관련 문헌 — Epic Games (UE 5.8) · Fowler (2021)
Gameplay Framework 및 Gateway 패턴을 참고한 State Ownership·Adapter 설계

03. Narrative Manager — Beat Graph

Beat를 노드로, 조건부 Transition을 간선으로 삼는 Directed Graph로 이야기 구조를 표현

Beat Graph

- Node = Beat
- Edge = 조건부 Transition



관련 연구 — Façade (Mateas & Stern, 2005), Story Beat 기반 서사 진행 구조 참고

Active Beat (현재 비트)

FIRST_CLUE

: 첫 단서를 공개하는 대화

Goal & Completion 조건

- **Goal** : *첫 단서 공개*
- **Completion** : *ClueRevealed == true*

상위 이야기 단위(=Parents) 참조 정보

- **Chapter** : *CHAPTER_01*
- **Sequence** : *SEQUENCE_01*

03. Narrative Manager — Narrative Context Assembly

현재 Beat를 기준으로 상위 Goal · Constraint를 참조해 Narrative Context 구성

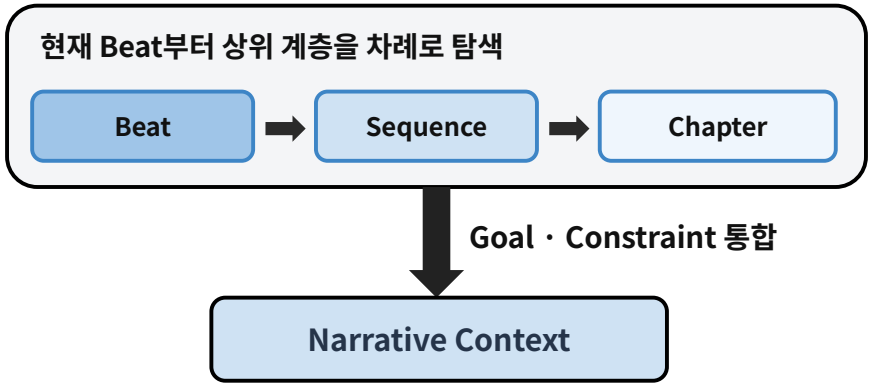
Narrative Hierarchy

: Chapter ⊃ Sequence ⊃ Beat



- 모든 단계: Goal은 필수, Constraint는 선택
- Beat: Completion 조건 필수

Narrative Context Assembly

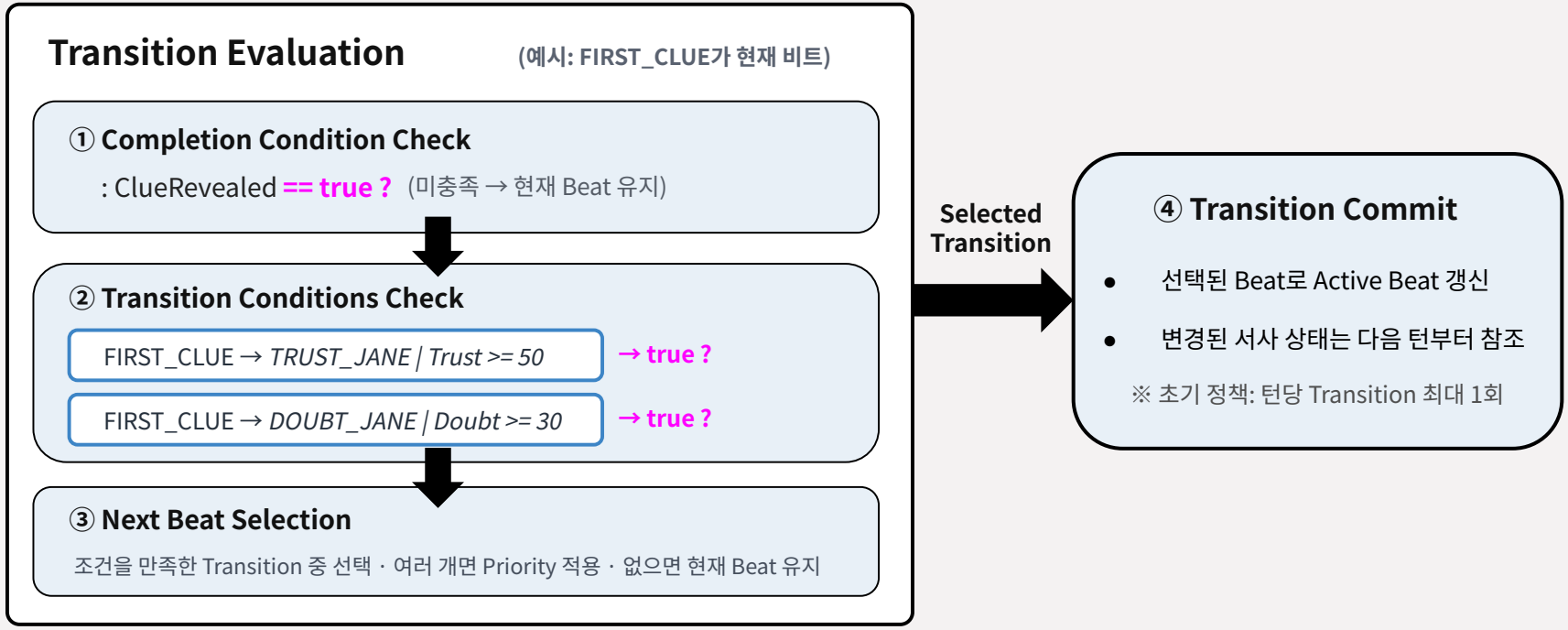


- **Goal ≠ Completion Condition**
  - Goal (자연어): '첫 단서를 공개한다' → LLM 응답 방향
  - Completion Condition (구조화): Flag == true → 완료 여부 판정
- Constraint: Soft = 자연어 지침 / Hard = 구조화 규칙 검증

관련 연구 — Façade (Mateas & Stern, 2005), Story Beat 기반 서사 진행 구조 참고

03. Narrative Manager — Beat Transition

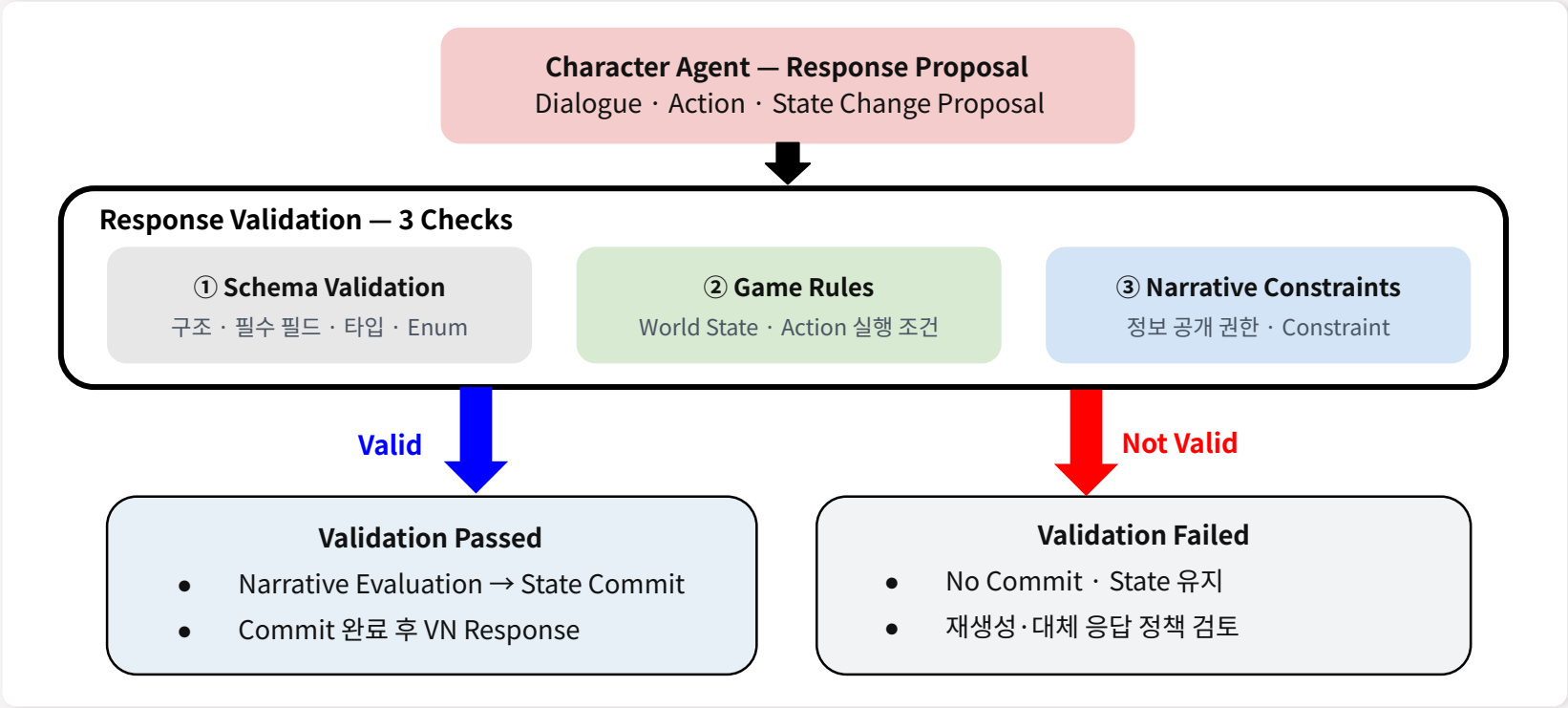
Beat의 Completion · Transition 조건을 평가하고, 선택한 Transition을 Narrative State에 반영



관련 연구 — Façade (Mateas & Stern, 2005), Story Beat · Drama Management 기반의 서사 진행 제어 개념 참고

Validation & Commit — 검증 및 상태 확정

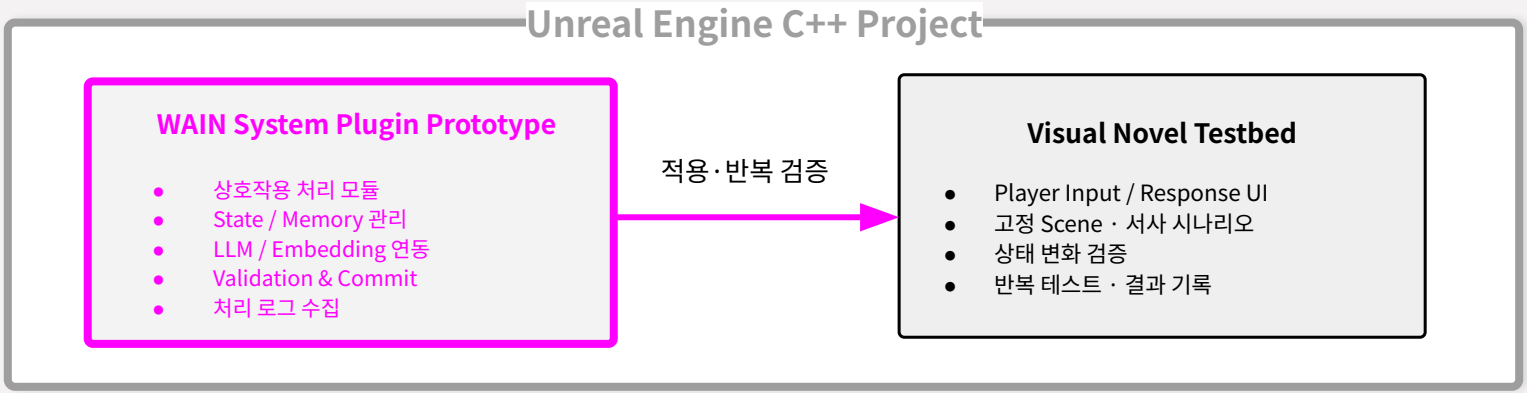
LLM Proposal의 구조·게임 규칙·서사 제약을 검증하고, 승인된 변경만 Commit



VI. 프로토타입 개발 및 테스트 계획

Prototype 구현 및 검증 환경

Unreal C++ Plugin을 VN Testbed에 연결하여 턴 처리·상태 변경·후속 턴 반영을 검증



주요 산출물

- WAIN System 플러그인 프로토타입
- 플러그인 소스 코드
- 시스템 설계 문서
- 테스트 및 평가 결과

- 프로토타입 검증 초점
- 단일 캐릭터·턴 기반 시나리오
- 입력 → 확정된 상태 변화 → 다음 턴 확인
- Input · Context · Validation · Commit 기록

구현 범위 및 한계 (Scope & Limitations)

프로토타입은 자유형 대화·상태 일관성·서사 제어 검증에 집중

Prototype Scope — 구현 범위

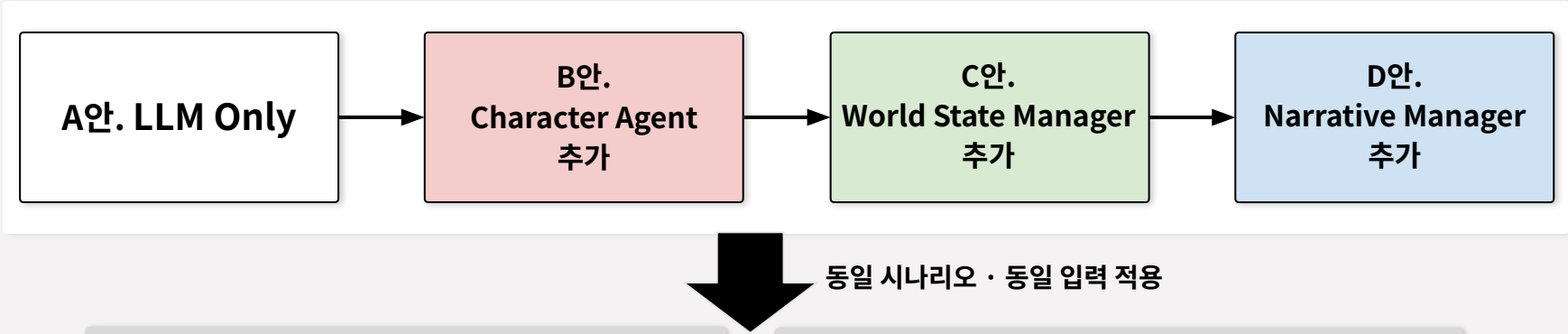
- Single-Agent · Turn-Based VN
- 자유형 대화 기반 응답 생성
- Memory · Planning 기반 Context 구성
- Validation · State Commit을 통한 상태 일관성 검증
- 조건 기반 Beat Transition

Limitations — 구현·검증 한계

- 제한된 Scene · 시나리오 중심 검증
- 실시간 World Events 미지원
- NPC 자율 행동 · Multi-Agent 실행 제외
- 실제 게임 시스템과의 통합 검증 제외

평가 및 검증 계획

동일한 테스트 시나리오와 입력에 시스템 구성 요소를 단계적으로 추가하여 각 기능의 효과를 비교



시스템 검증

- 기억 일관성 — 저장된 과거 경험을 이후 응답에 정확히 반영
- 관계의 연속성 — 누적된 관계 상태를 이후 반응에 지속적으로 반영
- 서사 준수 — 현재 Story Progress와 Narrative Constraint를 준수

테스트 시나리오 기반 자동 검증

기억 회수 정확도 · 상태 반영률 · 서사 제약 위반율 측정

응답 품질 평가

- 캐릭터 일관성 — 설정된 성격·가치관·말투를 지속적으로 유지
- 맥락 적절성 — 현재 입력과 대화 맥락에 적합한 응답 생성
- 자연스러움 — 대화 흐름과 캐릭터 반응이 자연스럽게 이어짐

평가자 기반 5점 척도

각 항목에 대한 응답 품질을 정성적으로 평가

VII. 추후 진행 일정

향후 개발 로드맵

시스템 기본 설계 완료 → Plugin / Agent 구현 → VN Testbed 통합 → 평가

주차	구분	개발 진행 내용
9주차	개발 기반	Plugin 구조·LLM API·공통 State 모델 정의
10주차	Agent 구현	Persona·Relationship·Emotion·Memory 구현
11주차	Agent 구현	Memory Retrieval·Reflection·Planning 구현
12주차	Manager 구현	World State Store·Schema·Adapter 인터페이스
13주차	Manager 구현	Narrative Graph·Goal/Constraint·VN 연동
14주차	통합·평가	Validator·Orchestrator 통합 및 턴 테스트
15주차	통합·평가	A~D 모듈 비교·상태 일관성·응답 품질 평가
16주차	최종 산출	프로토타입 시연·소스코드·기술 보고서 제출

최종 목표

- ▶ Unreal C++ Plugin · VN Testbed 완성
- ▶ Character·World·Narrative 상태 연계·검증
- ▶ 모듈별 기능 비교·평가 및 최종 산출물

감사합니다.

Appendix

— URD · SRS · FR / NFR 상세 명세

User Requirements Documents (URD)

프로젝트 목표에서 도출한 사용자 요구사항(UR)

ID	사용자 요구사항
UR-01	플레이어는 정해진 선택지 없이 자연어로 캐릭터와 자유롭게 상호작용할 수 있어야 한다.
UR-02	캐릭터는 과거 경험과 관계 변화를 기억하고 일관된 성격과 행동을 유지해야 한다.
UR-03	플레이어의 행동으로 변화한 게임 세계는 이후 상호작용에도 반영되어야 한다.
UR-04	자유로운 상호작용 중에도 서사 진행과 정보 공개 제약이 유지되어야 한다.
UR-05	게임 개발자는 WAIN을 다양한 VN 콘텐츠에 적용하고 기능을 검증할 수 있어야 한다.

※ 사용자 조사 결과가 아닌, 프로젝트 목표에서 도출한 요구사항 초안

Software Requirements Specification (SRS)

사용자 요구 및 설계 분석을 바탕으로 시스템 요구사항(SR) 구체화

구분	내용
Functional Requirements	FR-01~12 · 자연어 입력, 캐릭터 상태·기억, 세계·서사 제어, 응답 생성 및 상태 갱신
Non-Functional Requirements	NFR-01~07 · 모듈성, 상태 일관성, 확장성, 테스트 가능성, 관측 가능성, 안정성, 응답 성능
Interface Requirements	VN Testbed, LLM·Embedding Service, External State Adapter
Architecture Constraints	Unreal Engine C++ Plugin, Character / World / Narrative 역할 분리
Prototype Scope	Single-Agent · Turn-Based · 대화 기반 상태 갱신

※ SRS는 URD뿐 아니라 설계 분석에서 도출된 요구사항도 포함

Functional Requirements (FR) (1/2)

기능 요구사항(FR) 상세 명세 | FR-01~06

ID	기능 요구사항	담당 구성 요소
FR-01	자유형 자연어 Player Input 처리	WAIN System
FR-02	Character Persona 유지	Character Agent
FR-03	과거 상호작용을 Memory로 저장 및 검색	Memory
FR-04	과거 경험을 분석하여 상위 수준의 의미를 도출하고 후속 판단에 반영	Character Agent
FR-05	현재 Context와 경험에 따른 Planning	Character Agent
FR-06	Relationship / Emotion State 관리	Character Agent

Functional Requirements (FR) (2/2)

기능 요구사항(FR) 상세 명세 | FR-07~12

ID	기능 요구사항	담당 구성 요소
FR-07	게임 세계 상태의 조회 및 갱신	World State Manager
FR-08	현재 서사 상태에 따른 Goal / Constraint 결정	Narrative Manager
FR-09	캐릭터 대사 및 행동의 동적 생성	LLM
FR-10	Dialogue / Action / Expression / Event를 VN 응답으로 제공	WAIN–Testbed 연동
FR-11	상호작용 결과에 따른 State 갱신 및 다음 상호작용에 반영	State Management
FR-12	LLM 응답 및 행동 제안의 형식, 게임 규칙과 서사 제약을 검증하여 승인된 상태 변경만 적용	Response Validator / State Management

※ FR-12는 사용자 요구 UR-03, UR-04 및 설계 분석 과정에서 도출된 추가 요구사항

Non-Functional Requirements (NFR)

비기능 요구사항(NFR) 상세 명세 | NFR-01~07

ID	비기능 요구사항	검증 기준
NFR-01	모듈 독립성	Character / World / Narrative 역할 분리
NFR-02	상태 일관성	확정된 상태 변경이 이후 상호작용에도 유지
NFR-03	확장성	게임 콘텐츠와 독립적인 Plugin 구성
NFR-04	테스트 가능성	동일 시나리오로 모듈별 기능 비교 가능
NFR-05	관측 가능성	Input, 사용된 State, 응답 및 상태 변경 기록
NFR-06	안정성	LLM API 오류나 잘못된 응답 발생 시 처리 가능
NFR-07	응답 성능	입력부터 VN 응답까지의 지연시간이 허용 기준을 충족해야 함

※ NFR-07의 허용 지연시간은 초기 성능 측정 후 확정

Key References & Design Basis (1/2)

1. **Interactive Narrative: A Novel Application of Artificial Intelligence for Computer Games (Riedl & Bulitko, 2012)**
   - Interactive Narrative 및 플레이어 행동에 따른 서사 제어 개념 참고
   - **WAIN 반영:** Narrative Manager / Narrative Control 설계

2. **Generative Agents: Interactive Simulacra of Human Behavior (Park et al., 2023)**
   - Memory → Reflection → Planning 기반 장기 행동 생성
   - **WAIN 반영:** Character Agent의 Experience Adaptation 구조

3. **IBSEN: Director-Actor Agent Collaboration for Controllable and Interactive Drama Script Generation (Han et al., 2024)**
   - Director–Actor 역할 분리 및 Plot 제어 구조
   - **WAIN 반영:** Narrative Manager–Character Agent 역할 분리

Key References & Design Basis (2/2)

1. **Epic Games — Gameplay Framework in Unreal Engine (UE 5.8)**
   Martin Fowler (2021) — Gateway
   - Game-owned State 구조 및 Gateway 패턴 참고
   - **WAIN 반영:** World State Manager의 Ownership·Adapter 설계

2. **Mateas & Stern (2005) — Façade Interactive Drama Architecture**
   - Story Beat와 Drama Manager의 서사 제어 개념 참고
   - **WAIN 반영:** Narrative Manager의 Beat 기반 진행 구조

3. **Riedl & Young (2010) — Narrative Planning: Balancing Plot and Character**
   - Goal과 사건 간 인과 관계·캐릭터 의도 개념 참고
   - **WAIN 반영:** Narrative Goal·Constraint의 정합성 설계